

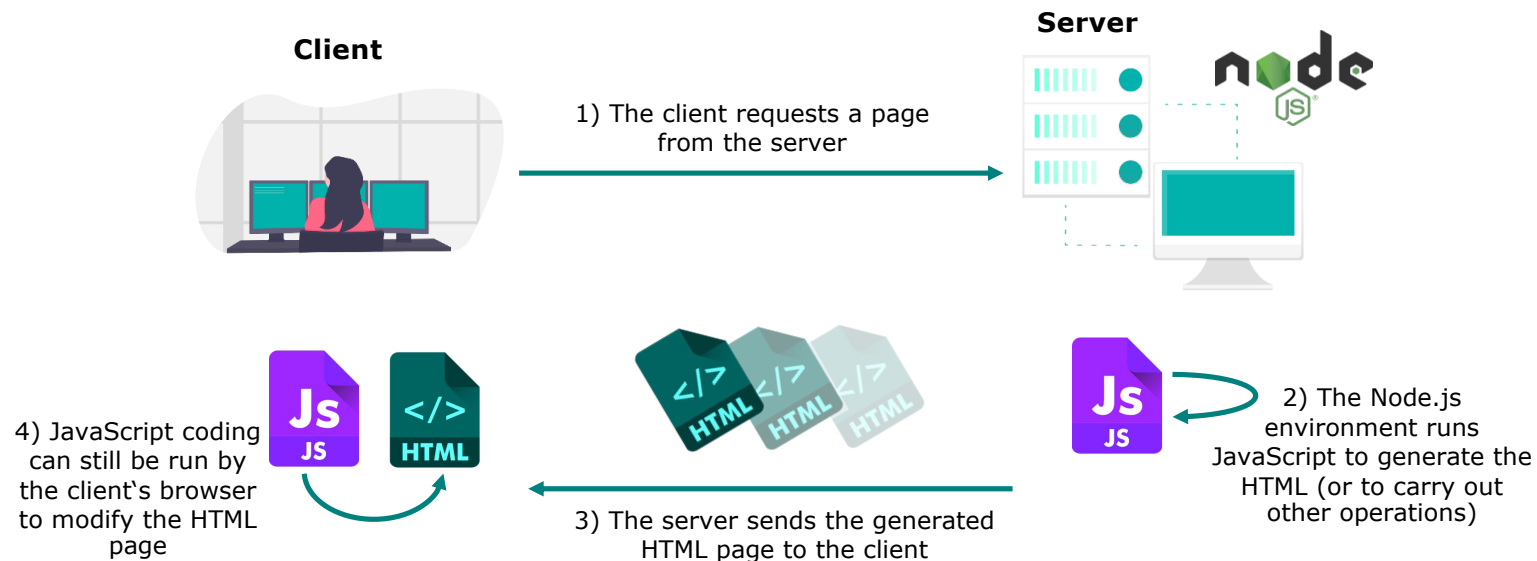


Node.js

Grundlagen, npm, Express.js

<https://nodejs.org/>

- Software, um JavaScript (**serverseitig**) auszuführen
 - > nur eine Sprache zur Entwicklung notwendig
 - gleiche Syntax, gleiche Mechanismen
 - agiler Einsatz möglich
 - > teilweise gleiche Bibliotheken für Server und Client!



- 2009: erste Veröffentlichung durch Ryan Dahl, gesponsert von Fa. Joyent
- 2010: Release von **express** (Web Application Server Framework)
- 2011: Einführung **npm** (Paketmanager für Javascript)
- 2015: Gründung Node.js Foundation
- 2020: npm wird von GitHub (Microsoft) gekauft
- 2026: aktuelle Version 25

- Plattform, um skalierbare Netzwerkanwendungen zu erstellen
 - Streaming
 - JSON-basierte REST-Dienste
 - Single-Page-Anwendungen
- basiert auf Googles JavaScript-Engine V8
- **eventbasiert, nicht-blockierend** (Event-Loop!!!)
- Opensource (MIT-Lizenz)
 - <https://github.com/nodejs/node>

Ihr Code ist der König

- Könige haben Heerscharen von Dienern
- Könige warten nicht gerne unnütz
- Morgens, wenn der König erwacht beginnt er mit seiner Arbeit und erteilt seinen Dienern Aufträge
- Anschaulich sind die Diener in node.js verborgene weitere Threads
- Der König hält sich hierbei nicht weiter mit den einzelnen Aufträgen auf und arbeitet weiter
- Wenn der König alles aktuelle erledigt hat schaut er nach ob einer der Diener bereits seine Aufgabe erledigt hat und bittet ihn darum zu berichten
- Natürlich kann immer nur ein Diener nach dem anderen dem König berichten, daher stellen sich alle Diener in einer Schlange an
- Der König beschäftigt sich also mit dem Bericht des aktuellen Dieners, wodurch ggf. weitere Aufträge an andere Diener erteilt werden
- Erst wenn der König den Bericht abgearbeitet hat kann der nächste Diener kommen

Es kann nur ein Diener nach dem anderen berichten!

```
var fs = require('fs'), sys = require('sys');

fs.readFile('eineDatei.txt', function(report) {
  sys.puts("Schau an was ich alle habe: " + report);
});

fs.writeFile('nocheineDatei.txt', 'etwas zu schreiben', function() {
  sys.puts("Und doch so viel zu tun");
});
```

Bei der Ausführung soll eine Datei gelesen und eine geschrieben werden.

Beide Aufträge werden ohne auf das Ergebnis zu warten (durch Aufruf einer speziellen Bibliothek die in separaten Threads Betriebssystem) getätigt.

Mit dem Aufruf wird eine anonyme Funktion mitgeliefert, die als callback-Funktion dient. Diese wird aufgerufen, wenn die aktuelle Abarbeitung fertig ist und die Aufgabe erledigt wurde.

Node.js wird aber immer nur ein Event nach dem anderen bekommen und verarbeiten, also immer nur einen Callback!

Die Event-Loop ist hierfür zuständig.

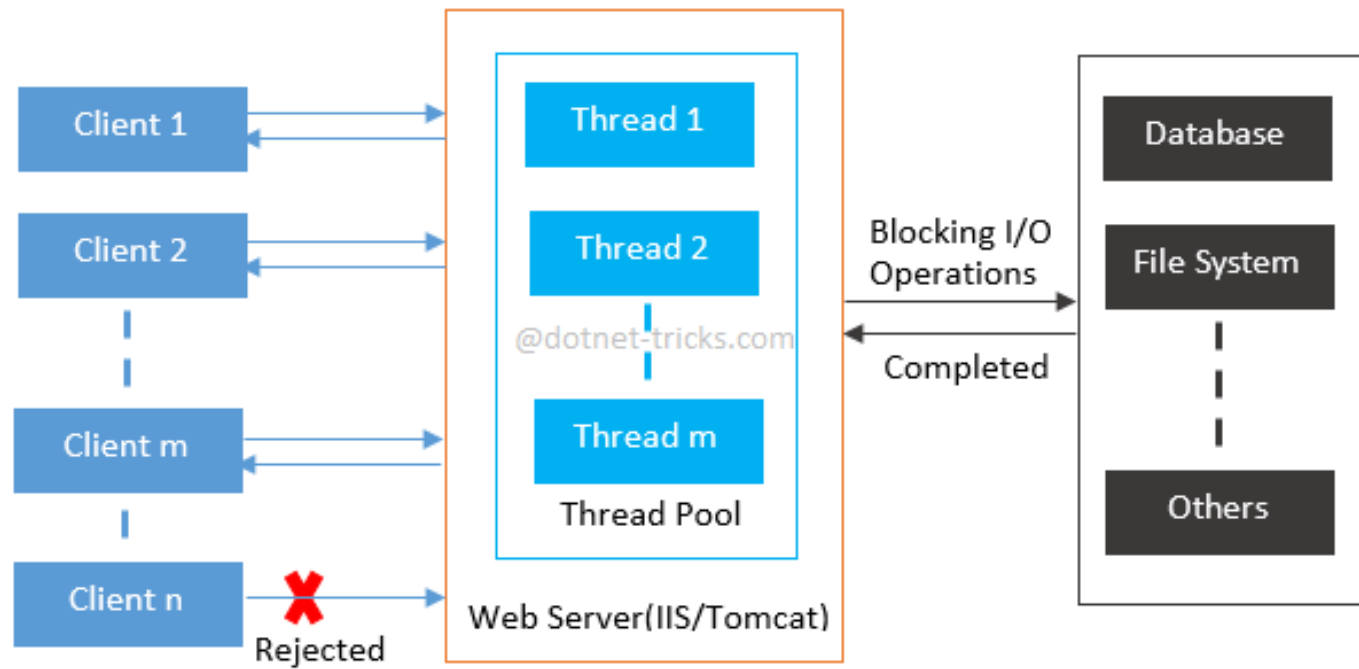
Node.js

threadbasiert vs. eventbasiert

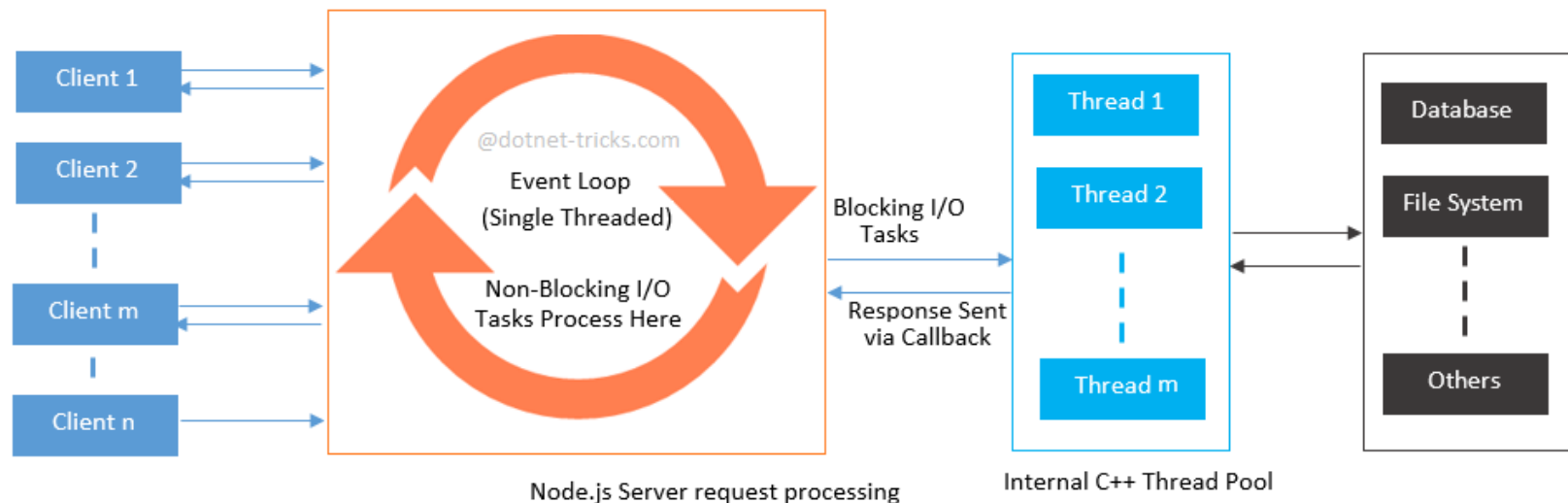
- Webserver sind normalerweise **thread- oder prozessbasiert**
 - Ein Thread oder Prozess pro Verbindung
 - Sinnvoll bei komplexeren Anwendungen mit viel Rechenlast
 - Problematisch bei sehr vielen Verbindungen, da dann sehr viele Threads/Prozesse gestartet werden (**C10K-Problem**)
- Node.js ist **eventbasiert**:
 - Aufträge, z.B. an das Betriebssystem, werden mit einem Callback versehen. Wenn die Aufgabe erledigt ist wird ein Event ausgelöst, der sich aber in einer Event Queue einreicht. Diese wird wenn nichts weiteres zu erledigen ist gemäß FIFO abgearbeitet
 - Server wartet nicht bis die Aufträge abgeschlossen sind, stattdessen wird das Programm weiter abgearbeitet.
 - **Events unterbrechen nicht, Sie reihen sich in eine Warteschlange ein**
 - In node.js verankerte Threads führen den Auftrag (blockierend) aus. Die Abgearbeitete IO-Operation wird im **Callback** dann abgearbeitet, wenn es in der Event-Queue an der Reihe ist
 - **asynchron + single threaded**

Node.js

<https://www.dotnettricks.com/learn/nodejs/nodejs-vs-other-server-side-frameworks>

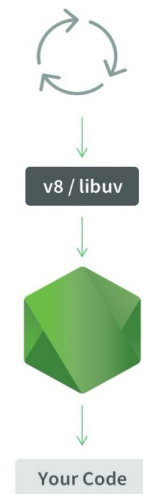


Multi-Threaded Web Server request processing

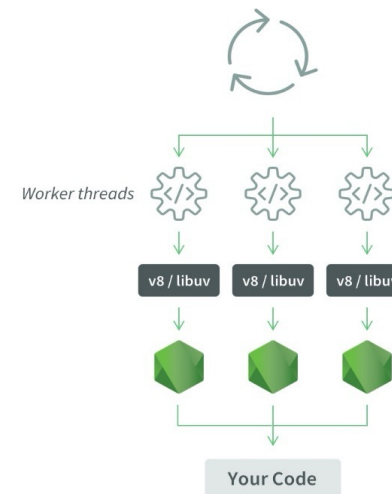


Die intern verwendete **libuv** stellt einen **Thread Pool** für blockierende Aufgaben bereit. **Dieser bestand standardmäßig aus 4 Threads (Bis Version 4)**. Aktuell ist der Default gleich der Anzahl der Kerne. Das Maximum sind 128 Threads

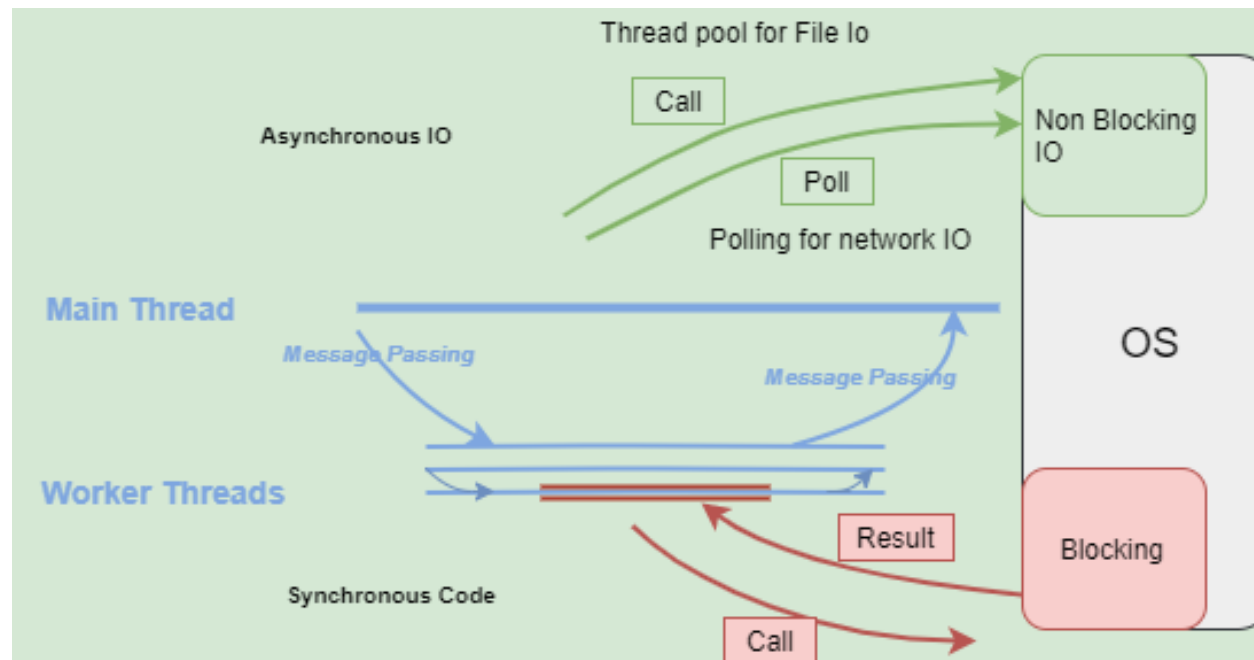
Standard Process Code



Process with Worker Threads



Tatsächlich gibt es mittels der Worker-Threads noch eine weitere Möglichkeit node.js zu skalieren. Anschaulich wird über das **Worker-Thread-Modul** eine eigene JavaScript Engine mit Event-Loop in einem separaten Thread gestartet. Die Kommunikation geschieht über **Message Passing** (postMessage und on-Methoden).



CPU-intensive Lasten sollte man über die libuv lösen oder in Worker-Threads auslagern: **Never block the Event-Loop**

Node.js

Beispiel: Daten von Datei lesen

- blockierend:

```
let data = fs.readFileSync („data.csv“);
```

- nicht-blockierend:

```
fs.readFile („data.csv“, function(err, data){  
    // do the processing here  
});
```

Hinweis:

Meistens geht man mit neueren Versionen dazu über, die **asynchronen Aufgaben** in eine **asynchrone Funktion** zu wrappen und mit `await` die folgenden Programmteile erst dann auszuführen, wenn man mit den vorher zu erledigen Aufgaben fertig ist. Für den Benutzer sieht es fast so aus, als ob an auf den Event/die Bedingung warten würde

(**async-await**/Promises kommen noch)

- Funktionalitäten, die in JavaScript-Dateien organisiert sind
 - Funktionen, Objekte, Variablen

Modularten

1. integrierte Module (Kernmodule)
 - müssen nicht installiert werden
2. lokale Module
 - Implementierung und Nutzung eigener Module
3. Third Party Module
 - Einbindung per Node Package Manager

Anwendungsschwerpunkte Node.js dynamische Webanwendungen

- File System
- HTTP(s)
- Path
- URL
- OS
- ...

Node.js

Third Party Module

Node Package Manager (npm)

- vielfältiger Projektmanager für Node.js
 - Konfiguration über `package.json`
 - Metainformationsverwaltung (Name, Beschreibung, Lizenz, ...)
 - Verwaltung des Development-Zyklus
 - Starten des Projekts
 - Testen des Projekts
 - Bauen des Projekts
 - ...
 - Abhängigkeitsverwaltung von [Third Party Modulen](#)
 - installieren
 - updaten
 - Versions-Management
- Veröffentlichung eigener Module im npm-Repository

- wichtige Kommandozeilen-Befehle:
 - `npm init`
 - erzeugt interaktiv (mit Benutzerfragen) eine `package.json`, welche Informationen über das node.js Programm sowie alle erforderlichen Abhängigkeiten inkl. der Versionen enthält
 - `npm install <package name>`
 - Paket wird geladen und in das Unterverzeichnis `node_modules` des Projekt-Ordners installiert und in der `package.json` aufgenommen
 - `npm install`
 - installiert alle in `package.json` angegebenen Abhängigkeiten in den lokalen `node_modules` Ordner (ohne `-g` Option ist dies im aktuellen Verzeichnis und damit nur dort)
 - `-g` braucht üblicherweise Admin-Rechte (außer man nutzt einen Mac mit Homebrew)
- weitere npm Befehle: <https://docs.npmjs.com/cli-documentation/>

Node.js

Beispiel package.json

```
{
  "name": "demo_server",
  "description": "description",
  "authors": "v.sander@fh-aachen.de",
  "version": "1.0.0",
  "main": "index.html",
  "repository": {
    "type": "git",
    "url": "https://git.fh-aachen.de/demo_server.git"
  },
  "license": "SEE LICENSE IN /LISCENCE.MD",
  "dependencies": {
    "bootstrap": "^4.1.1",
    "jquery": "^3.3.1",
    "mustache": "^2.3.0",
    "express": "^4.16.4"
  }
}
```

Relevant bei Veröffentlichung, sonst eher "private": true

Version ab der Kompatibilität gewährleistet ist

Default ist die npm-registry
<https://registry.npmjs.org>

Node.js

Wichtige Third Party Module

- **Express**
 - serverseitiges Webframework
 - einfaches realisieren von Reaktion auf URL-Anfragen (Routing)
 - unser Standardeinstieg auf eine “GET /“-Anfrage des Browsers ist app.js (z.B. durch Start mit node app.js)
- **Sequelize**
 - ORM für SQL-basierte Datenbanken
 - einfache Persistenzschicht
- **Mustache**
 - Template-Engine
- **Bootstrap**
 - Frontend-Framework für responsives Design
- **JSDoc**
 - API-Dokumentations-Generator für Javascript

Node.js

Einbinden von Modulen (**alte Methode**)

- Einbindung per **require**
- Beispiel 1:
 - Nutzung des integrierten Moduls `path`:

```
// app.js  
const path = require('path')
```

- Beispiel 2:
 - Nutzung des 3rd-Party-Moduls `express`

```
// app.js  
const express = require('express')
```

- **ACHTUNG!** Muss vorher per `npm` installiert worden sein (da 3rd-Party)!
- Hinweis: `require` sucht zunächst die integrierten Module und verwendet dann die Umgebungsvariable `NODE_PATH` um ggf. in verschiedenen Verzeichnissen nach der angegebenen Datei zu suchen. Die Endung `.js` ist optional!

Node.js

Anwenden eigener Module (**alte Methode**)

- Funktionalität eines Moduls muss aufgrund der Gültigkeitsregeln den Nutzern explizit zur Verfügung gestellt werden: **exports**
- Beispiel 3:
 - **Exportieren einer anonymen** Funktion in einem Modul:

```
// hellworld.js
module.exports = function () {
  console.log('bar!');
}
```

- **Nutzung** der anonymen Funktion in der Applikation:

```
// app.js
const hello = require('./helloworld.js');
hello();
```

- Beispiel: Ein solches Vorgehen kann bei der Verwendung des Express-Moduls genutzt werden

```
// app.js
var express = require('express');
var app = express();
```

Hinweis: `// app.js`
`var express = require('express');`
`var app = express();`

Das hier gezeigte Vorgehen ist ein typisches Muster in Javascript, bei dem im Hintergrund das bereits bekannte Konzept der Closures verwendet wird!

Node.js

Einbinden von Modulen (**alte Methode**)

- Beispiel 4:
 - Exportieren einer **benannten** Funktion in einem Modul:

```
// helloworld.js - Hier ohne module. - dafür mit .hello
exports.hello = function () {
    console.log('bar!');
}
```

- **Nutzung** der benannten Funktion in der Applikation:

```
// app.js
const helloworld = require('./helloworld.js');
helloworld.hello();
```

- Hinweis: **bei exports muss** im Gegensatz zu module.exports immer **ein Bezug** (Properties: Klasse, Attribut, Funktion, ...) mit **angegeben werden**. Export ist genau für diesen Bezug da
- Exports regelt den Zugriff auf explizite „Properties“ (mehrere mit export {p1, p2, ...} möglich oder mehrere exports)

- **Neue Möglichkeit** zur Einbindung von Modulen über import
- Import läuft asynchron und passt damit gut zu Promises (kommen noch)
- ES-Module haben die Endung .mjs außer
- bei Verwendung: `node --experimental-modules main.js`
- **Einfache Alternative:** `package.json "type" : "module"`
- **Import vereinfacht die Verwendung von Modulen deutlich**
 - Erlaubt die einfache Integration spezifischer Variablen und Objekte (**Named Imports**)
 - Verbesserte Kapselung, require würde eigentlich auf dem globalen Scope arbeiten und vermeidet etwaige Konflikte durch eine Kapselung über eine interne (anonyme) Funktionen, die sofort aufgerufen wird (*Immediately-Invoked Function Expression*). Ein ES-Module-File mittels import/export wird vom JavaScript-Parser bereits in einen **eigenen lexical-Scope** gehüllt
 - Folgt dem Konzept des asynchronen Ladens und blockiert damit nicht, require blockiert

<https://www.geeksforgeeks.org/difference-between-node-js-require-and-es6-import-and-export/>

Node.js

Einbinden von Modulen (**alter, komplizierter Weg**)

- Beispiel 5:
 - Exportieren eines Objektes in einem Modul

```
// Message.js  
exports.SimpleMessage = 'Hello World!'
```

oder:

```
module.exports.SimpleMessage = 'Hello World!'
```

- Nutzung des Objektes in der Applikation:

```
// app.js  
const msg = require('./Message.js')  
console.log(msg.SimpleMessage)
```


Node.js

Schreiben von Modulen (**alter Weg**)

`module.exports` VS. `exports`

- `exports` ist ein Alias von `module.exports`

```
// cat.js
class Cat {
  makeSound() {
    return 'Meowww';
  }
}
module.exports = Cat;
//exports.Cat = Cat;
```

Bei Klassen besser
`module.exports`
verwenden!

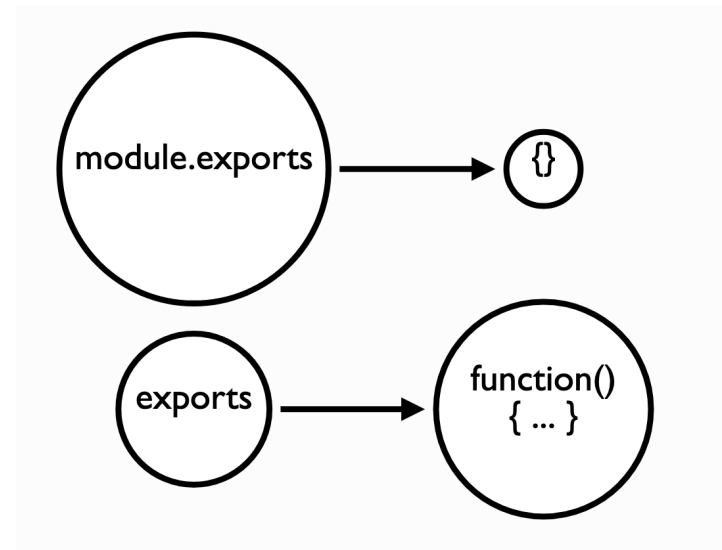
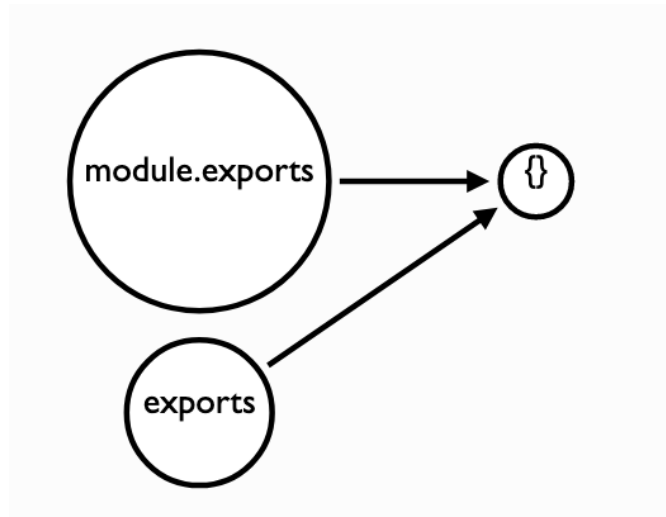
```
// app.js
const Cat = require('./cat.js')
const cat = new Cat();
//const cat = new Cat.Cat(); !
console.log(cat.makeSound())
```

geht so auch,
aber unschön

Node.js

Aufpassen beim Einbinden von Modulen (alter Weg)

- Beide sind Referenzen auf das gleiche (leere) Objekt:
- Bei Veränderungen laufen diese auseinander



<https://blog.tableflip.io/the-difference-between-module-exports-and-exports/>

Node.js

Verbesserte Einbindung von Modulen

- Mittels ES6 gibt es durch die **import/export-Systematik** eine deutlich verbesserte Möglichkeit
 - Exportieren eines Objektes in einem Modul

```
// Message.js
export const SimpleMessage = 'Hello World!'
export function sayHello(name) {
  console.log(`Hello, ${name}!`)
}
```

- Nutzung des Objektes in der Applikation:

```
// app.js
import { SimpleMessage as msg, sayHello } from ('../Message.js')
console.log(msg)
sayHello('Volker')
```

Der Pfad wird hier
sogar automatisch
aufgelöst



Node.js

Verbesserte Einbindung von Modulen

```
// MeineKlasse.js
export default class MeineKlasse {
  constructor(name) {
    this.name = name;
  }
  sayHello () {
    console.log(`Hello, ${this.name}!`);
  }
}
```

Bei default-Export wird genau das eine Element (die Klasse) exportiert. Das import kann ohne Spezifikation eines Namens (also ohne {}) erfolgen

- Nutzung des Objektes in der Applikation:

```
// app.js
import MeineKlasse from './MeineKlasse';
const einObjekt = new MeineKlasse('Volker');
einObjekt.sayHello();
```

Hinweis:

Closures wurden auch zur Kapselung privater Daten in JavaScript so dominant. Das Einbinden von Modulen mit `require` bedingte dies und führte zu einem Pattern mit einer anonymen Selbst-aufrufenden Funktion (IIFE-Wrapper). Mit `import` `let` und `const` ist diese Notwendigkeit nicht mehr da.

Allerdings bleiben Closures trotzdem in JavaScript weiterhin unverzichtbar

- Rückgabe von Funktionen die später auf interne Zustände zugreifen müssen
- **Callbacks** die auf Variablen des umgebenden Scopes zugreifen
- Mehrfachinstanzen mit Statusvariablen

Node.js

einfachste Node.js Anwendung

- Node.js beinhaltet eine Skript-Engine für Javascript
 - funktioniert ähnlich wie Python, PHP, ...
 - Skripte werden oben nach unten abgearbeitet
 - Aufpassen: Das Vorgehen ist zumeist asynchron und nicht blockierend

```
// demo.js
function concat(a,b){
    return a+b;
}
let a = "Javascript ist";
let b = " aus dem Web nicht
mehr wegzudenken";
console.log(concat(a+b));
```

Applikation starten:

```
node demo.js
```

Output:

```
Javascript ist aus dem
Web nicht mehr
wegzudenken
```

Was wollen wir eigentlich?

- Daten bereitstellen
- Webseite bereitstellen (Frontend)
- Dienste (Services) bereitstellen (Backend)
- Web-Applikation (Frontend + Backend) bereitstellen

Klassisch

Anwendungsfall	Software (Beispiel)
Daten bereitstellen	Apache (htdocs) / php (Datenbank)
Webseite bereitstellen	Apache (htdocs)
Dienste (Services) bereitstellen	Tomcat → Java Servlet /php

Node.js: Ereignisgesteuert

- Node.js implementiert im **http-Modul** einen Webserver
- jede HTTP-Anfrage löst bei Node.js ein Ereignis aus
- Funktionen reagieren auf die Anfragen und verarbeiten die Daten (hier **handleRequest**)
- Einbindung des Moduls: `import http from 'http';`
- Nutzung des Moduls: `let server = http.createServer(handleRequest);`
- Starten des HTTP-Servers mittels `server.listen`-Methode, die faktisch nur das listen und nicht das accept macht:

```
server.listen(3001, function() {  
    console.log("Server listening on:  
                http://localhost:„ + port);  
});
```


Node.js: erste Server-Applikation (app.js)

```
import http from 'http';
```

Server-Port

```
// port  
const port = 3001;
```

Request
Funktion

```
// request handle function  
function handleRequest(request, response){  
  response.end('request URL: http://localhost:  
  + port + request.url);  
}
```

create
Server

```
// create the server  
const server = http.createServer(handleRequest);
```

start
Server

```
// start the server  
server.listen(port, function(){  
  console.log("Server listening on: http://localhost:" + port);  
});
```

Server im Terminal starten:

```
node purenodeserver.js
```

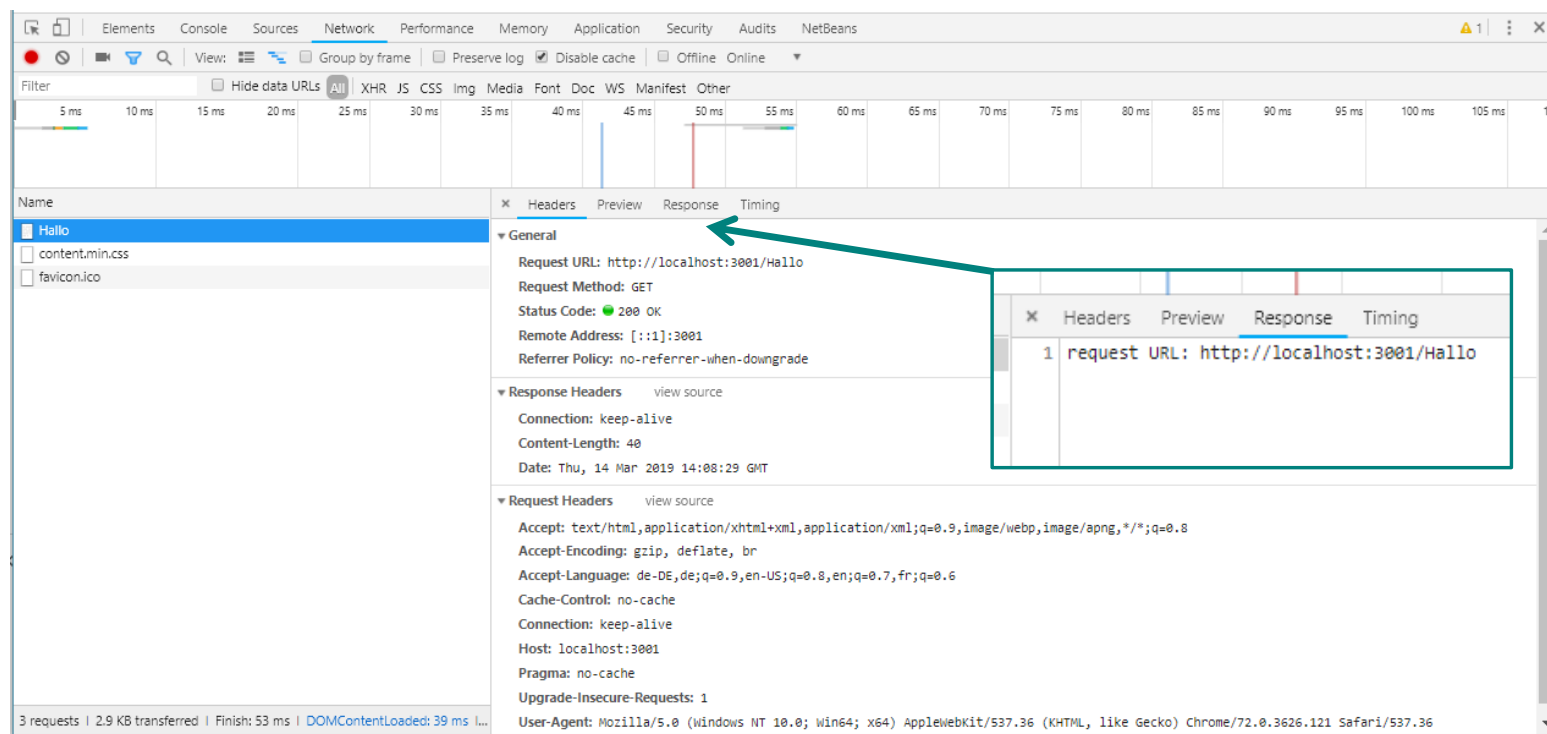
Zugriff auf Server via Web-Browser:

```
http://localhost:3001/Hallo
```

Browser-Ausgabe:

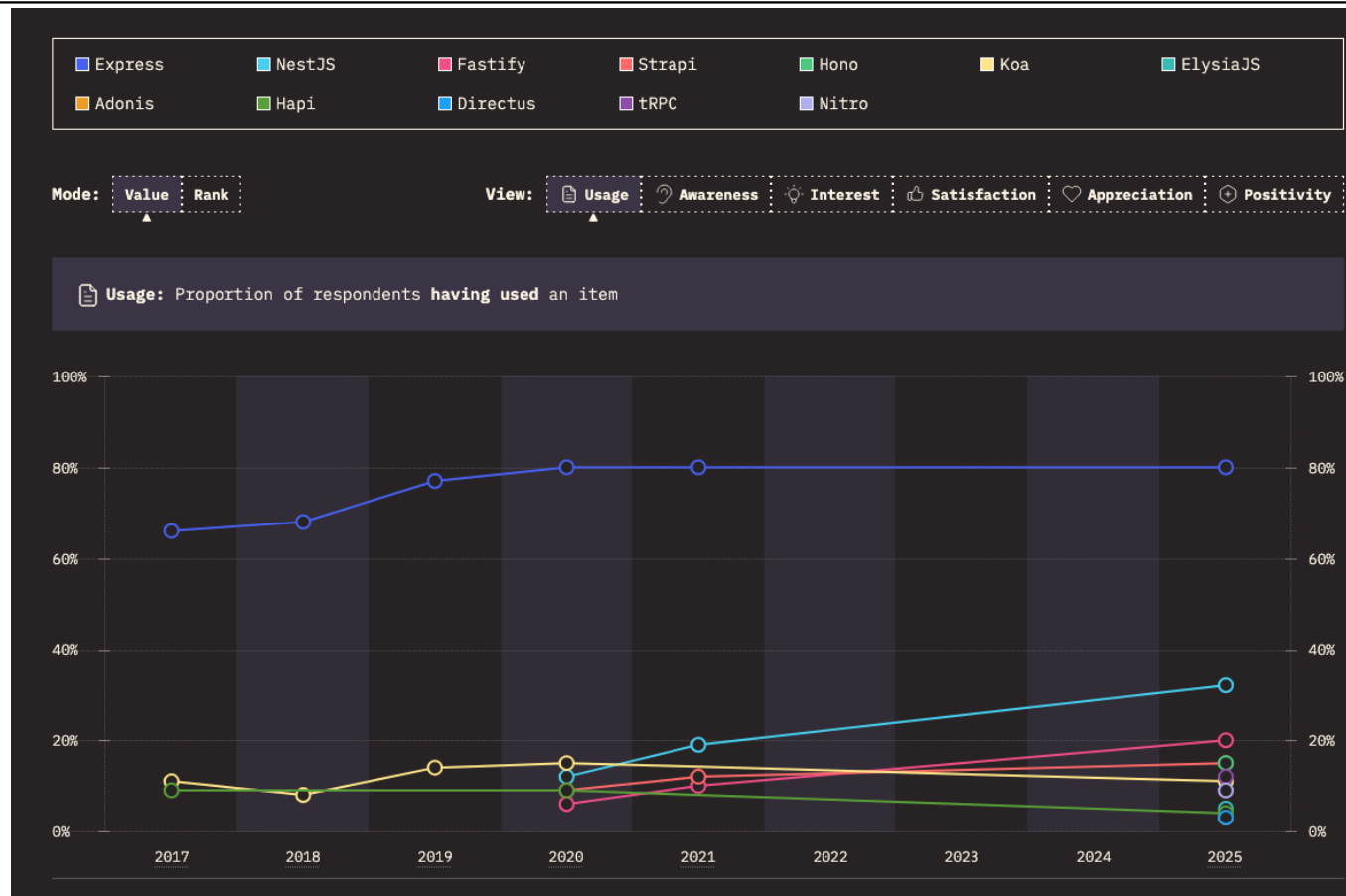
```
Request URL: http://localhost:3001/Hallo
```

Browser-Developertools (F12)



Node.js

Backend-Frameworks – StateOfJS 2025



<https://2025.stateofjs.com/en-US/libraries/back-end-frameworks/>

Express.js

- Meistgenutztes Web-Framework für Node.js
 - Einbindung: `require('express')`
 - `import express from('express')`
- Open Source, MIT-Lizenz
- Funktionalitäten
 - Routing
 - Middleware-Module
 - Ausliefern von statischen Dateien
 - Cookies und Session Manipulation
 - ...



Express.js

- Läuft „hinter“ dem Node.js HTTP Server
- Express wird faktisch als Modul (Middleware) mit in die Bearbeitungskette der HTTP-Anfragen in node.js eingebunden

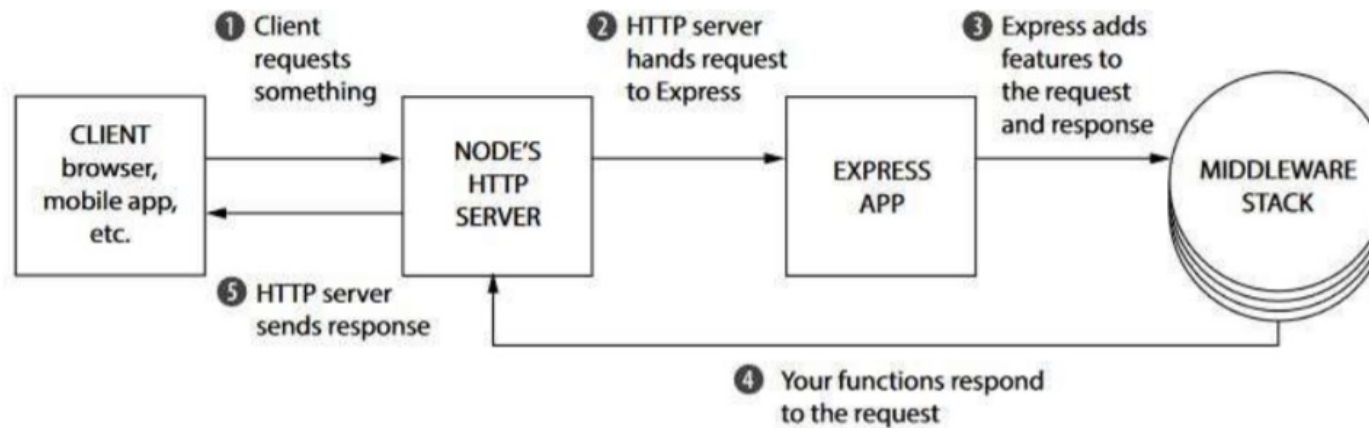


Abb.-Quelle: Evan M. Hahn: Express in Action - Writing, building, and testing Node.js applications, Manning Publications, 2016

vanilla Node.js (app.js)

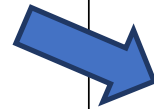
```
import http from 'http'

// port
const port = 3001;

// request handle function
function handleRequest(request, response){
  response.end('Hello World');
}

// create the server
const server =
http.createServer(handleRequest);

// start the server
server.listen(port, function(){
  console.log("Server listening on:
  http://localhost:" + port);
});
```



Express (app.js)

```
import express from ('express');
const app = express();

app.get('/', function(req, res){
  res.send('Hello World');
});

app.listen(3000, function(){
  console.log('Example app
  listening on port 3000!');
});
```

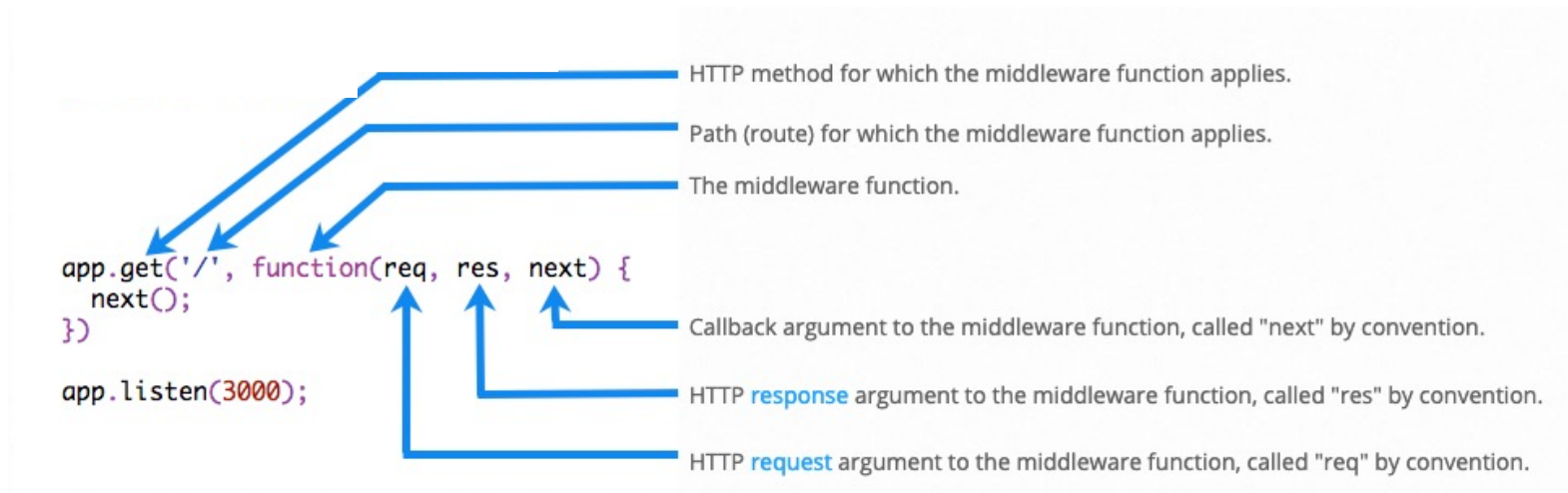
Route



Hinweis: app.get bezieht sich auf eine HTTP-GET-Operation auf der angegebenen URL! app ist hier das aktivierte express-Modul

Express.js

<https://expressjs.com/en/guide/writing-middleware.html>



Was sind Routen?

- Navigation innerhalb der Anwendung
- setzen sich zusammen aus der HTTP-Methode und der URL
 - URL:
`http://localhost/myModule/subRoutine`
 - zugehörige Route (bei `app.get`):
`GET /myModule/subRoutine`
 - das gilt genau nur für die Route (oder die zutreffenden Routen)
- **Eingehende HTTP-Methode + relativer Pfad (Route)** wird auf eine Callback-Funktion abgebildet:

```
// express
app.get('/myModule/subRoutine', function(req, res){
    // ...
});
```

Express.js

Routen

`app.js` (unser Einstiegspunkt der Applikation) sollte bei größeren Projekten eigentlich keine Implementierungsdetails der Routen enthalten

→ daher: Auslagerung der Routen abhängig von der Funktionalität in Dateien

→ Einbindung der Routen als Module in `app.js` (`require`)

Beispiel:

```
//routes.js
import express from 'express'
const router = express.Router();
router.get('/', (req, res) => {
    res.render( ... )
})
export { router };

//app.js ruft route.js auf
import express from 'express'
import { router } from './routes.js'
const app = express();
app.use('/', router);
```

Hier wird eine Javascript Datei aufgerufen (template), üblicherweise im views-Verzeichnis

Merke: Routen werden **First-Come-First-Serve** interpretiert. Der erste im Codeverlauf zutreffende Eintrag wird zur Bedienung der Anfrage genommen.

Damit verschiedene Module gestaffelt genutzt werden können gibt es das **Middleware-Konzept** in Express. Hierdurch kann eine **Aufrufkette** realisiert werden!

Problem: Routen enthalten Strukturinformationen (absolute URLs). Code nicht wiederverwendbar/schwerer zu parametrisieren

Lösung: Router:

- Sammlung von Routen zur besseren Organisation der Anwendung
- **Sub-Pfade** können einfach definiert und **von den Routen getrennt** werden
→ ermöglichen eine Wiederverwendbarkeit der Module

- ohne Router:

```
app.get('/wichtigeURL/first', ...  
app.get('/wichtigeURL/second', ...  
app.get('/wichtigeURL/third', ...
```

- mit Router:

```
const router = express.Router();  
router.get('/first', ...  
router.get('/second', ...  
router.get('/third', ...  
export = { router }
```

```
//app.js  
...  
app.use('/wichtigeURL', router)
```

```
const router = express.Router();
router.get('/first', ...
router.get('/second', ...
router.get('/third', ...
export { router }

//app.js
...
app.use('/first', router)
```

`app.use` ist wichtig, denn es bezieht sich auf ein Konzept in express:

Middleware!

In unserem Fall gilt: Wir binden die router-Funktion (Middleware) ein (mit all ihren get-Defintionen und setzen dabei den Pfad (/wichtigeURL) auf den sie wirkt!

Express.js

Router Beispiel

```
import express from 'express';
```

```
const router1 = express.Router();  
const router2 = express.Router();
```

```
app.use('/Products', router1);  
app.use('/Customers', router2);  
router1.get('/', function(req, res){  
    res.send('Products');  
});
```

```
router2.get('/', function(req, res){  
    res.send('Customers');  
});
```

```
router2.get('/Count', function(req, res){  
    res.send('50 Customer');  
});
```

```
app.listen(3000, function(){ console.log('Example app  
listening on port 3000!'); });
```

Router

URL Festlegung
für die
Middleware

Products/

Customers/

Customers/Count

Express.js

Router Beispiel - Refactoring

```
import express from 'express';
const router2 = express.Router();

router2.get('/', function(req,
res){...});
router2.get('/Count', function(req,
res){...});

export { router2 };
```

```
import express from 'express';
const router1 = express.Router();

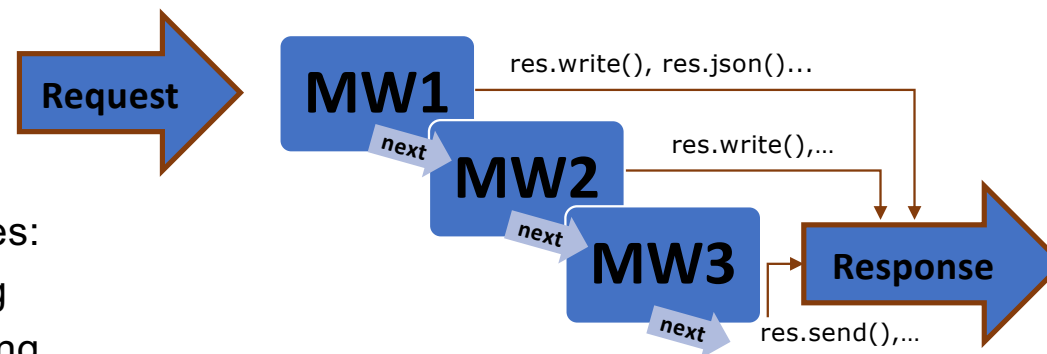
router1.get('/', function(req,
res){...});

export { router1 }
```

```
import express from 'express';
const app = express();
import { router1 } from './Routes/router1.js';
import { router2 } from './Routes/router2.js';
app.use('/Products', router1);
app.use('/Customers', router2);
app.listen(3000, function(){ console.log('Example app
listening on port 3000!'); });
```

Was sind Middlewares?

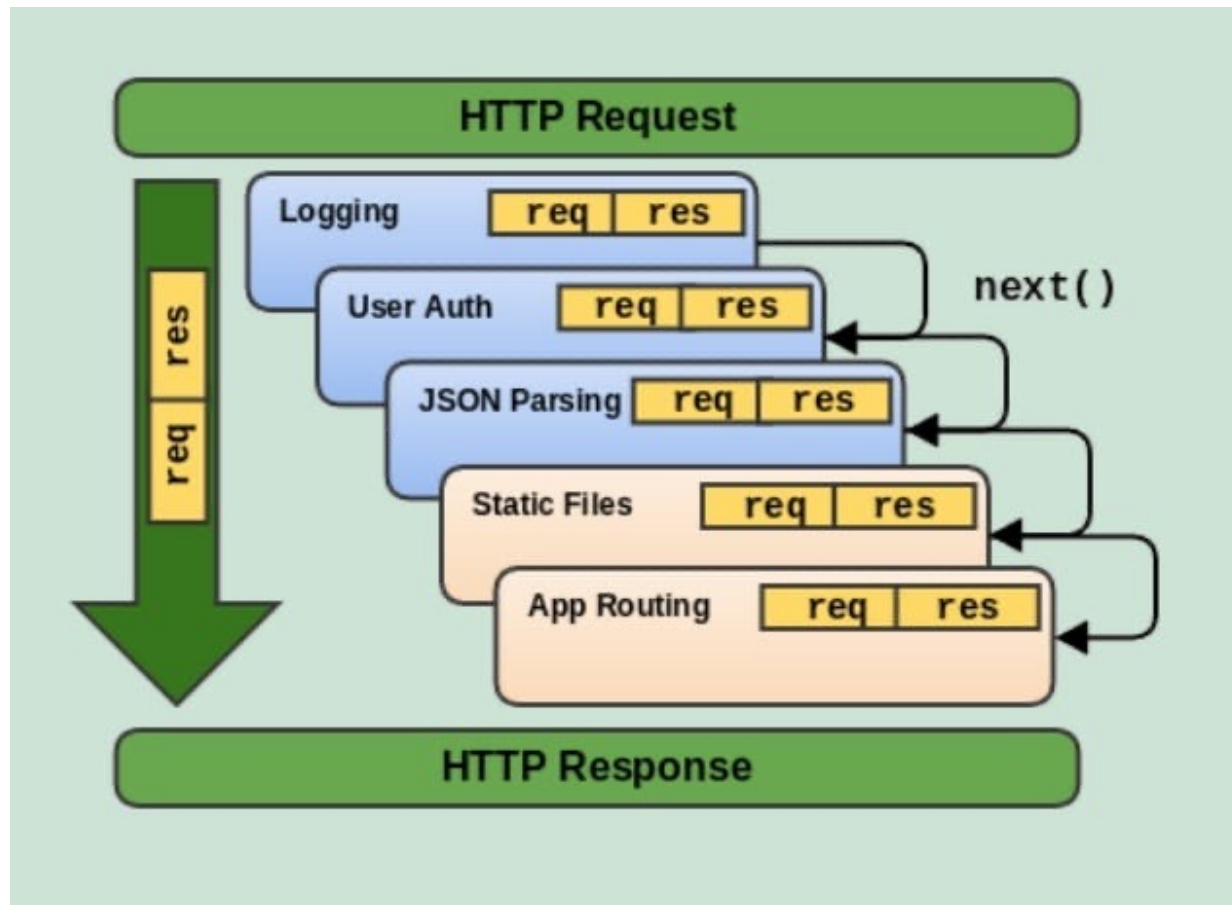
- Stack von Funktionen mit beliebigem Inhalt
- werden zwischen dem Eingehen eines Requests und dessen Bearbeitung durch einen Handler geschaltet
- haben Zugriff auf Request, Response und nächste Middlewarefunktion (über **Variable next**)
- **Aufpassen**: `res.send()` schließt und verschickt die Nachricht



- Beispiele für Middlewares:
 - Parsing, Autorisierung
 - Logging, Error Handling
- **Verwendung** der Middleware unabhängig von Methode per `app.use`

Express Middleware

<https://dev.to/ghvstcode/understanding-express-middleware-a-beginners-guide-g73>



Express Middleware

<https://expressjs.com/en/guide/routing.html> -> Syntax mit require

```
const express = require('express') —————> Besser import express from 'express';  
const router = express.Router()
```

```
// middleware that is specific to this router  
router.use((req, res, next) => {  
  console.log('Time: ', Date.now())  
  next()  
})  
// define the home page route  
router.get('/', (req, res) => {  
  res.send('Birds home page')  
})  
// define the about route  
router.get('/about', (req, res) => {  
  res.send('About birds')  
})  
  
module.exports = router
```

birds.js

```
const birds = require('./birds')  
  
// ...  
  
app.use('/birds', birds)
```

Express.js

Middlewareares

```
import express from 'express';  
const app = express();  
const user = 'Volker';  
const password = '1234'
```

```
function middleHandler(req, res, next){  
  console.log(req.originalUrl);  
  next();  
}
```

Definition
einer
Middleware

```
app.use('/',function(req, res, next){  
  // Authorization test  
  let query_user = req.query.user;  
  let query_pw = req.query.pw;  
  if(query_user === user && query_pw === password){  
    next();  
  }else{  
    res.send('Zugriff verweigert');  
  }  
});
```

Eigene
Middleware
für alle Routen

```
app.get('/', middleHandler, function(req, res){  
  res.send('Zugriff gestattet');  
});  
app.listen(3002);  
console.log('start server on port 3002');
```

Benutzung zweier
Middleware
Beendet durch
Middleware ohne next

Express.js

Aufruf mit GET

```
import express from 'express';  
const app = express();  
const user = 'Volker';  
const password = '1234'
```

```
function middleHandler(req, res, next){  
  console.log(req.originalUrl);  
  next();  
}
```

Dann das

```
app.use('/',function(req, res, next){  
  // Authorization test  
  let query_user = req.query.user;  
  let query_pw = req.query.pw;  
  if(query_user === user && query_pw === password){  
    next();  
  }else{  
    res.send('Zugriff verweigert');  
  }  
});
```

Erst wird das
ausgeführt

```
app.get('/', middleHandler, function(req, res){  
  res.send('Zugriff gestattet');  
});  
app.listen(3002);  
console.log('start server on port 3002');
```

Zuletzt das

Express.js

Aufruf mit POST

```
import express from 'express';  
const app = express();  
const user = 'Volker';  
const password = '1234'
```

```
function middleHandler(req, res, next){  
  console.log(req.originalUrl);  
  next();  
}
```

```
app.use('/', function(req, res, next){  
  // Authorization test  
  let query_user = req.query.user;  
  let query_pw = req.query.pw;  
  if(query_user === user && query_pw === password){  
    next();  
  }else{  
    res.send('Zugriff verweigert');  
  }  
});
```

Nur das wird
ausgeführt

```
app.get('/', middleHandler, function(req, res){  
  res.send('Zugriff gestattet');  
});
```

```
app.listen(3002);  
console.log('start server on port 3002');
```

[app.METHOD](#)(<pfad>,<middleware>) und [app.use](#)(<pfad>,<middleware>)

Mögliche Angaben für den Pfad:

- Eine String-Darstellung des Pfades
- Ein Pfad Muster (Path Pattern, Untermenge von regulären Ausdrücken)
- Ein regulärer Ausdruck
- Ein Array mit einer Kombination der angegebenen Möglichkeiten

Unterschiede zwischen **app.METHOD (Routing von Anfragen)** und **app.use (Einbindung von Middlewares)**:

- **Bei app.use() gilt der Pfad auch für Subpfade**
 - `app.use('/', ...)` gilt für `'/'` aber auch für `'/products'`
- `app.METHOD()` unterstützt [Pfadparameter](#) (Route parameters)
 - `app.get('/products/:id', ...)` gilt für `/products/42`, dabei ist der Wert `"42"` über `req.params.id` auslesbar (**Achtung** ist immer ein String)
 - Der Pfadparameter ist hier exakt gemeint und gilt ohne reguläre Ausdrücke nicht rekursiv

Express ermöglicht den Aufbau einer Verarbeitungskette durch Middlewares

- `app.METHOD()` bindet einen Handler (ggf. Kette) also Middleware **für eine Route** und **eine bestimmte Methode** (z.B. GET, POST, PUT, PATCH, DELETE) ein
- `app.use()` bindet einen Handler (ggf. Kette) also Middleware für **alle Anfragen an einen Pfad und alle Subpfade** (ohne Pfad heißt `'/'`). Anschaulich ist das ein **Mount** der Middleware in dem Pfad
- Die Reihenfolge der Einbindung im Source-Code bestimmt die Ausführungsreihenfolge
- Ohne Aufruf von `next()` bricht die Kette ab!

- `app.all('*')` entspricht `app.use('/')`
- **Vorsicht!**
- `app.use('/api')` springt bei den URLs `/api` und `/api/resource` an, `app.all('/api/*')` aber nur bei `/api/resource` und nicht `/api`

Express.js

Routen - Middlewares

```
import express from 'express';
const app = express();
app.all('/api/*', function(req, res, next) {
  console.log('only applied for routes that begin with /api`)
  next()
})
```


Hinweis:

Die Middlewares senden ihre Antwort oft mittels `res.send()` oder `res.json()`

Diese Aufrufe schließen den Antwort-Stream, so dass keine weiteren Ausgaben, z.B. in anderen Middlewares möglich sind.

`res.write()` wird vom `http`-Modul geerbt und schließt den Stream nicht. Hier kann in den Middlewares beispielsweise der HTTP-Header gesetzt werden

```
res.set('Content-Type', 'application/json');  
res.write(JSON.stringify(obj));  
res.end();
```



Erzeugt ein JSON-String

```
res.set({ 'Content-Type': 'text/plain',  
          'Content-Length': ,123  
});
```

Hinweis:

Wenn `res.send()` oder `res.json()` den Antwort-Stream schließen, warum gibt es aber die `res.end()` -Methode überhaupt?

Wenn Sie keine Antwort verschicken wollen, dann ist `res.end()` die Lösung

```
res.status(404).end();
```

express - das Response Objekt

Responses können auf unterschiedliche Art und Weise gebaut werden:

Methode	Beschreibung
<code>res.download()</code>	Gibt eine Eingabeaufforderung zum Herunterladen einer Datei aus.
<code>res.end()</code>	Beendet den Prozess "Antwort".
<code>res.json()</code>	Sendet eine JSON-Antwort.
<code>res.jsonp()</code>	Sendet eine JSON-Antwort mit JSONP-Unterstützung.
<code>res.redirect()</code>	Leitet eine Anforderung um.
<code>res.render()</code>	Gibt eine Anzeigevorlage aus.
<code>res.send()</code>	Sendet eine Antwort mit unterschiedlichen Typen.
<code>res.sendFile</code>	Sendet eine Datei als Oktett-Stream.
<code>res.sendStatus()</code>	Legt den Antwortstatuscode fest und sendet dessen Zeichenfolgedarstellung als Antworthauptteil.

<https://expressjs.com/de/guide/routing.html>

Hinweis: Die Antwort wird bei einigen automatisch beendet. Außerhalb der express-API gibt es auch noch `res.write()`, bei dem dies nicht der Fall ist.

express - das Response Objekt

redirect

`res.redirect([status,] path)`

Beispiel 1: Logout

```
router.get('/logout', (req, res) => {  
  req.session.destroy((err) => {  
    if(err) return console.log(err)  
  });  
  res.redirect('/')  
});
```

Beispiel 2: Weiterleitung von einer alten URL auf die neue URL:

```
app.get('/old-url/', (req, res, next) => {  
  res.redirect(301, 'https://domain.com/new-url')  
  
})
```

Express.js

Template-Engines

```
//routes.js
import express from 'express'
const router = express.Router();
router.get('/', (req, res) => {
    res.render( ... )
})
export { router };
};
```

alle Express-konformen
Template-Engines exportieren
eine Funktion, die über `res.render()`
indirekt aufgerufen wird und so
Vorlagen in HTML überführt

```
//app.js ruft route.js auf
import express from 'express'
import { router } from './routes.js'
const app = express();
app.use('/', router);
```

Wir nutzen hier also ein zusätzliches Modul (Template-Engine),
um die Antwort (HTML) über Vorlagen (Templates) zu erzeugen

Damit wir die Vorlagen so auch nutzen können, müssen gewisse Einstellungen passen

- require/import auf die verwendete Template-Engine
- Festlegen der Engine in Express für das render:

```
import express from 'express';  
import mustacheExpress from 'mustache-express';  
app.engine('html', mustacheExpress());  
app.set('view engine', 'mustache');
```

Ggf. Festlegen des Verzeichnisses mit den Vorlagendateien (Default ist Verzeichnis /views). Beispiel:

```
app.set('views', 'templates');
```